ECE 332 – Embedded Systems Lab

Lab 4: Hand-Written digit recognition using DE1-SoC

Objectives

- Part 1: Train your neural network on your host machine
- Part 2: Use your trained weights to perform inference on DE1-SoC board

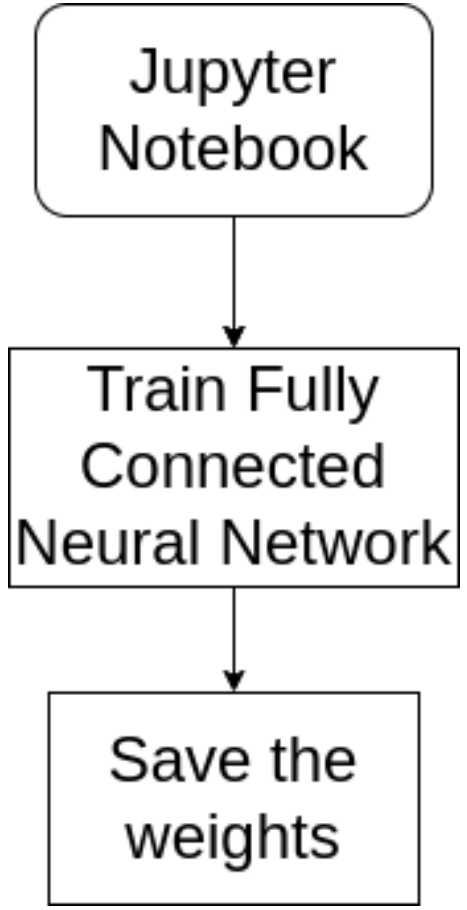
Lab Components

- Neural Network Training
  - Jupyter Notebook: Skeleton code for training a single-layer fully connected neural network. Guidance on how to train and save weights.
- Inference
  - DE1-SoC Board: Utilization of previously implemented OpenCL kernel from Lab 3. C code development for launching kernels and handling synchronization.

Part 1 : Train Neural Network using Jupyter notebook

- Skeleton code provided to you

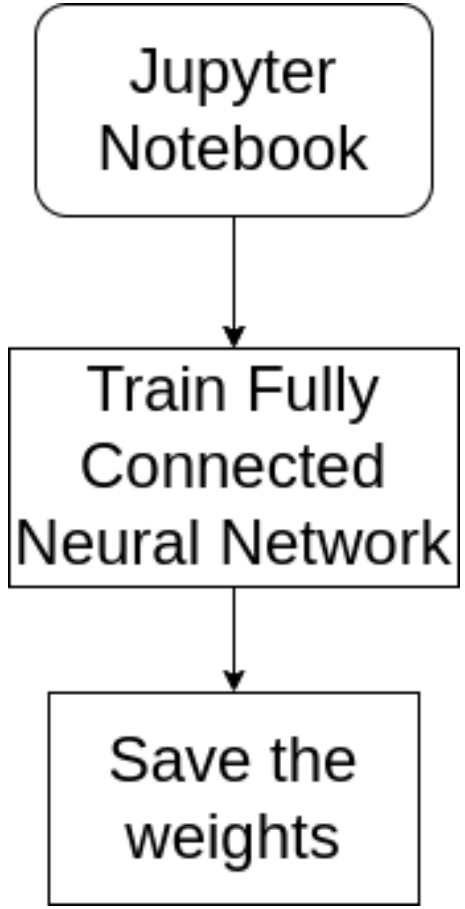
- Define the Neural Network Architecture
  - fully connected layers
  - Size of each layer
  - Number of layer
  - Don't use Convolution Layers, this will require significant coding in C on DE1-SoC



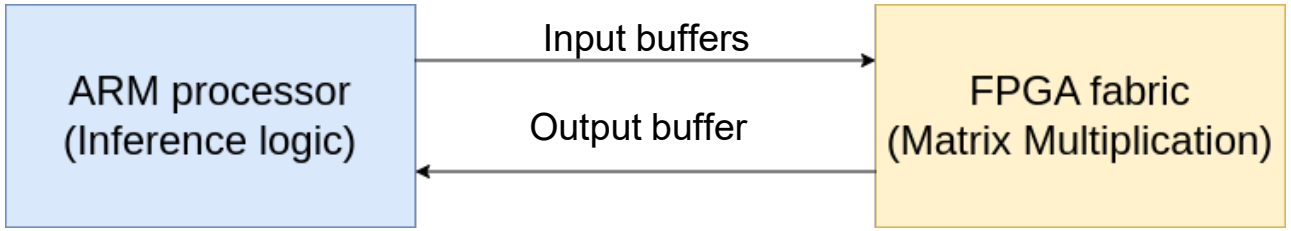
Part 1 : Train Neural Network using Jupyter notebook

- Output : bin files for each layer that we will load into our c program and use it for inference

- Network architecture used for taining should be same as the one that we are going to implement on DE1-SoC



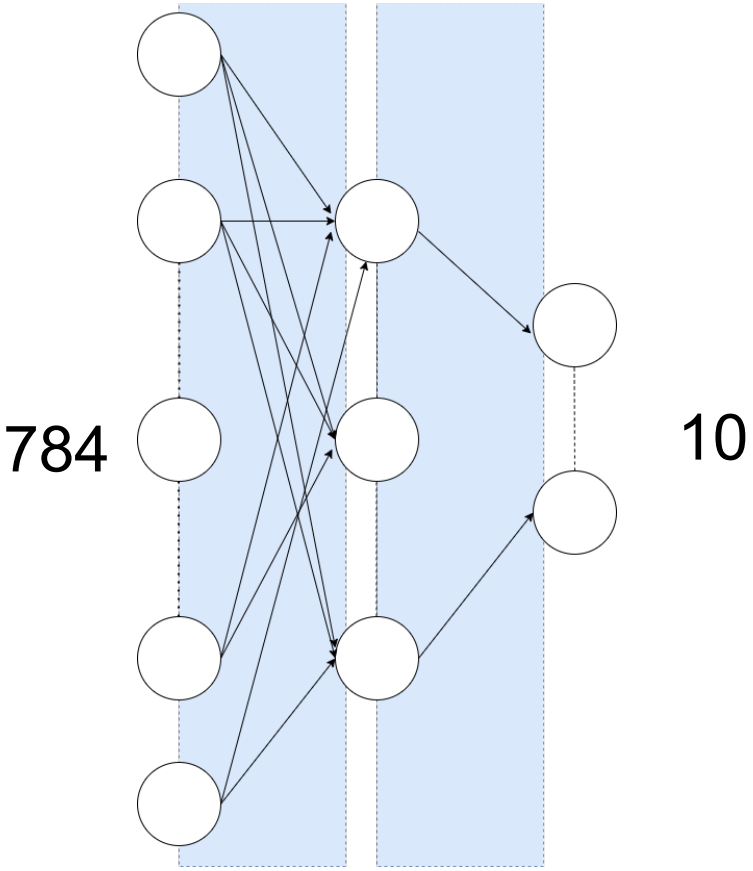
Part 2 : Inference on DE1-SoC



- Implement Layers in C
  Load buffers
- Launch Kernels
- Read buffers

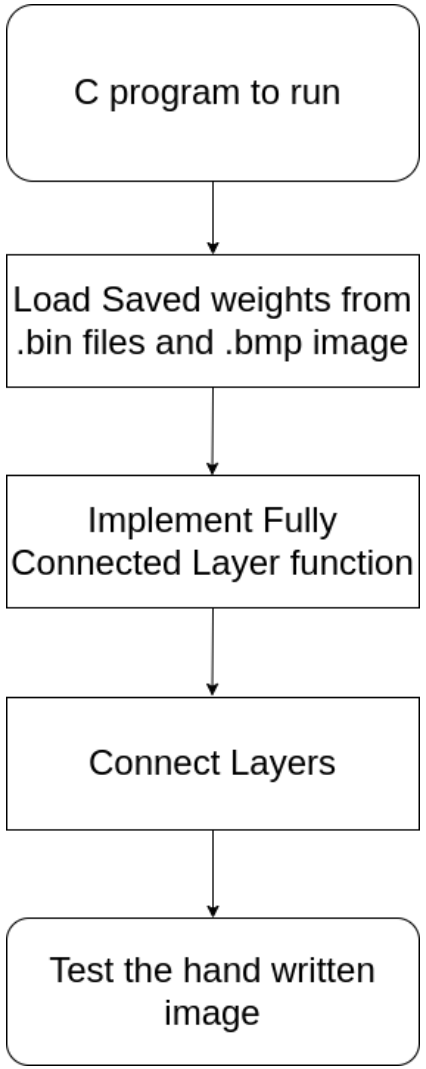
Part 2 : Inference on DE1-SoC

- Input : 28 x 28 pixel image
- 10 output labels (10 digits)
- We perform the computations highlighted in blue



Part 2 : Inference on DE1-SoC

- Inputs
  - bin files you generated from part1
  - capture a bmp image, holding a digit, using the Lab 3 code
  - test this with image with python first (testing code for python will be provided to you)
- Output
  - predicted label (number) for the image

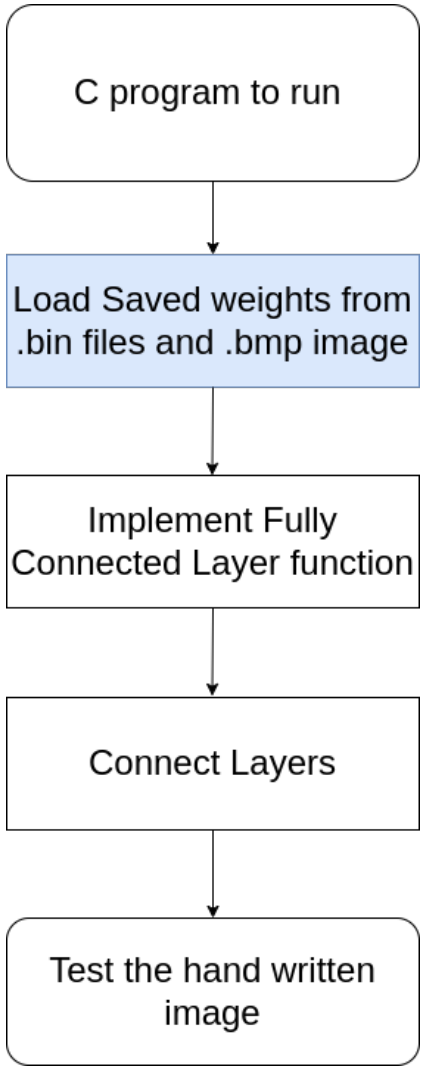


Part 2 : Inference on DE1-SoC

- Functions provided to load .bin file weights and .bmp image (black and white image)
- Capture the image of a hand-written digit using capture image code from Lab 3
- Test that image using the python testing code provided to you
- Load the bmp file using the function provided to you

Tasks:

1. Capture image, test the black and white image using python code on your system
2. Load .bin weights and .bmp image into C code, functions are already provided

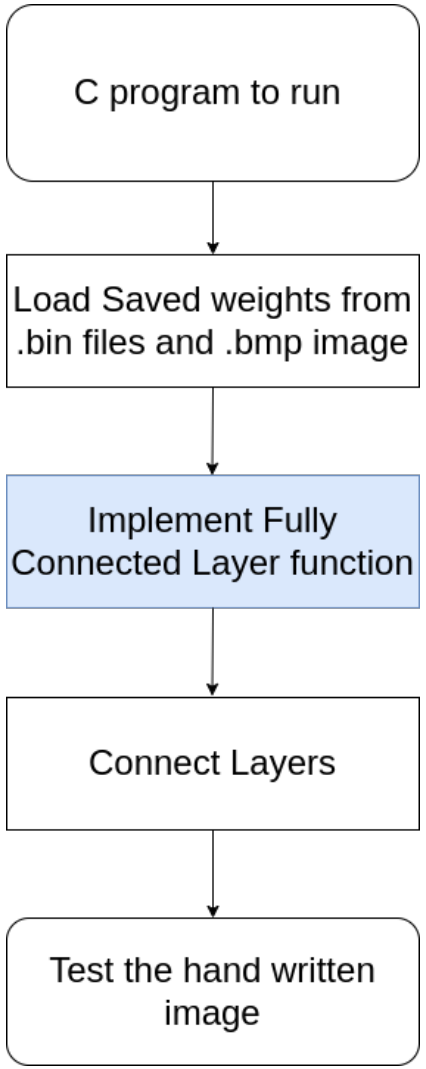


Part 2 : Inference on DE1-SoC

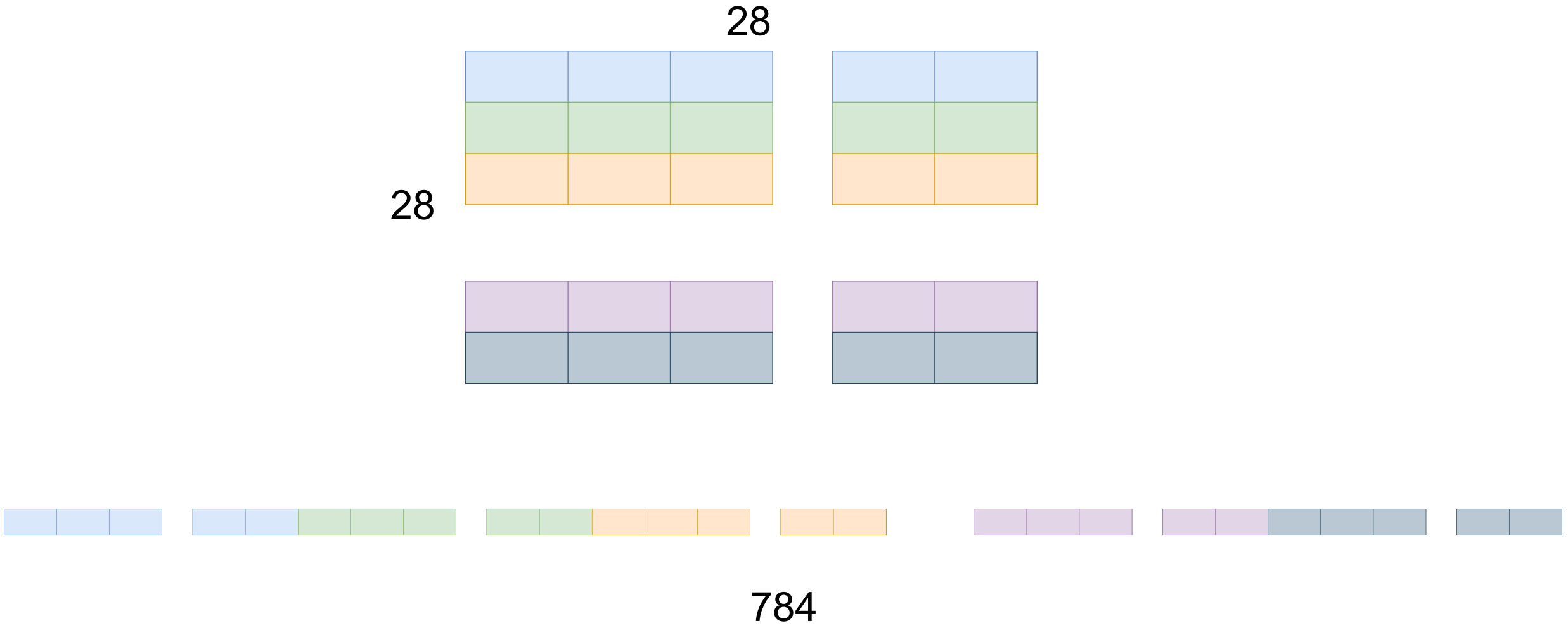
- Implement the Fully connected layer
- What does Fully connected layer do?
  - Multiply weights with incoming inputs
- Implement the tiled matrix multiplication
- Why tiled matrix multiplication?
  - Cannot fit whole matrices into FPGA memory and perform operations using the PEs (Processing Elements)

Tasks:

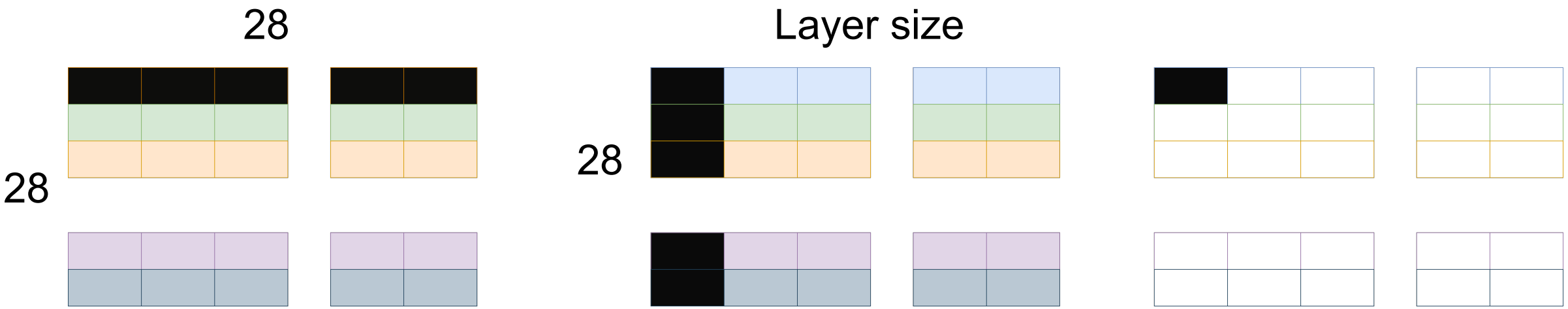
1. Implement logic for tiled matrix multiplication



Images and weights in 1D array format



Tiled Matrix Multiplication



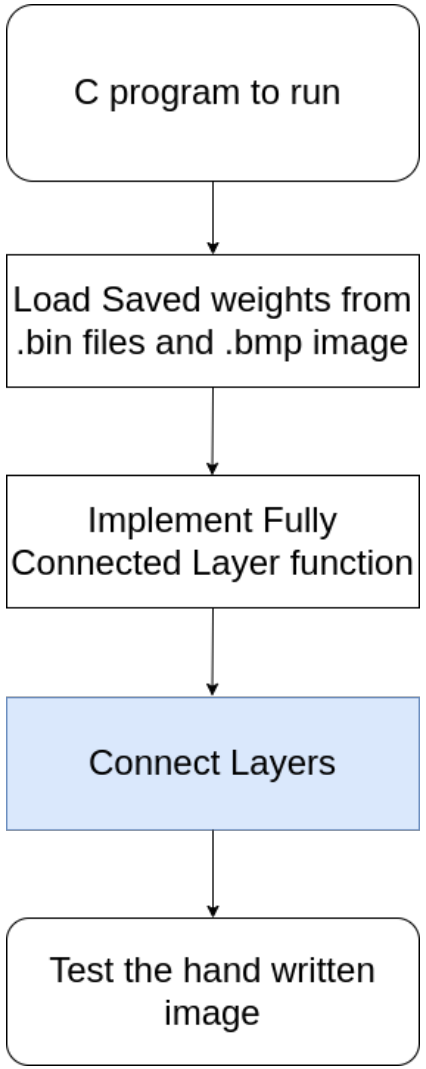
Part 2 : Inference on DE1-SoC

- Layer function will be finished once tiled matrix multiplication is implemented

Tasks:

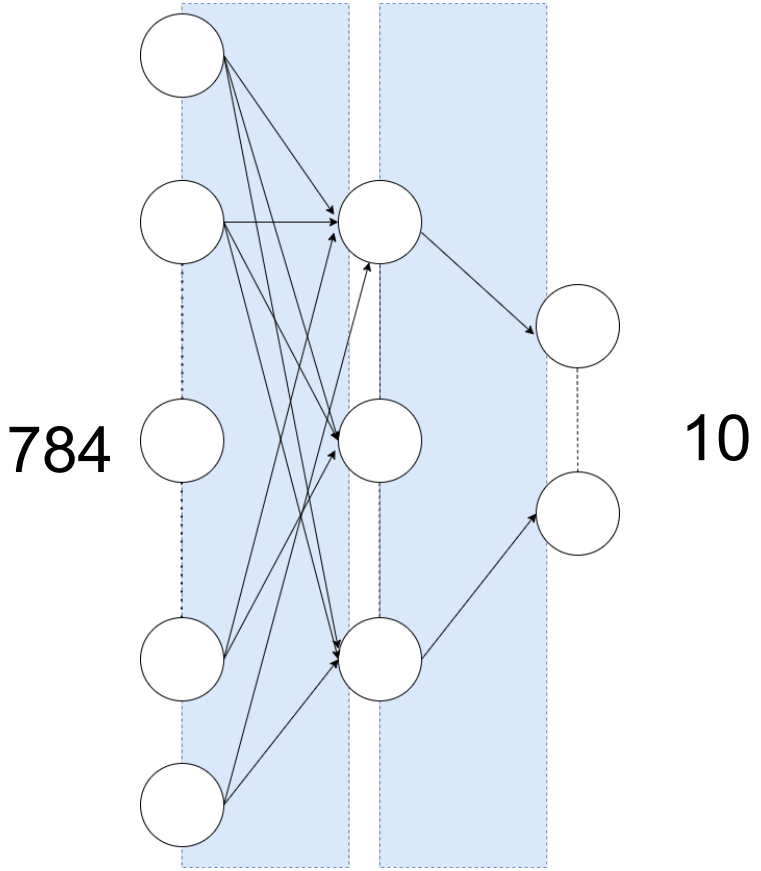
1. Connect layers : Call the layer function based on the neuron size. If you have a single hidden layer, you have to call the function twice with input and weight sizes passed as arguments

2. In between you need to call relu and softmax functions (which is already provided to you)



Connect Layers

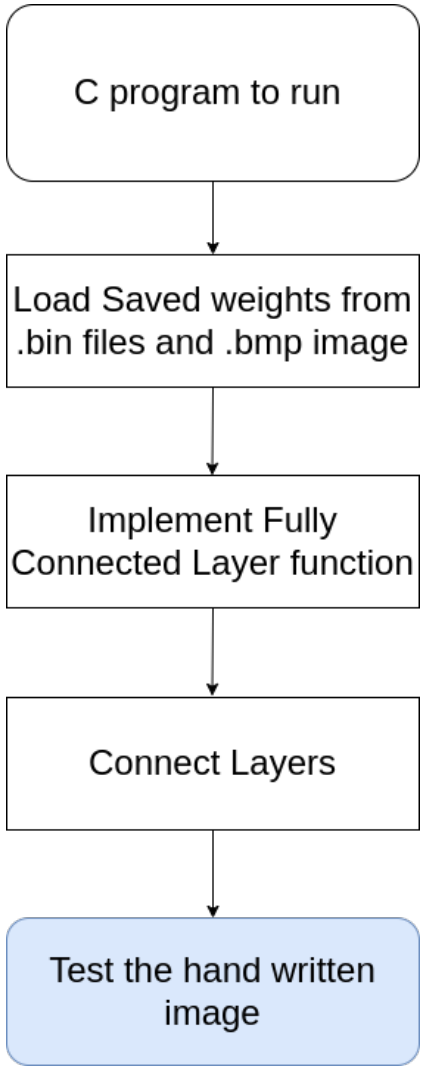
- For single hidden layer, function calls in general in the order
  - Layer
  - ReLU (if the previous layer is hidden layer)
  - Layer (output layer)
  - Softmax(if the previous layer is output layer)



Note : We are performing softmax and ReLU on ARM CPU and these functions are provided in the base code

Part 2 : Inference on DE1-SoC

- After completing above steps and running the code, it should print the predicted label on to the terminal



Testing purpose

- As capture_image.c in lab3, you will run the code on Linux DE1-SoC board
- But if you want to use your lab machine and better GUI like VScode, you can first test these functions solely on your host machine and then integrate these functions on to the actual code
- Instructions for this and the base code for performing this on your machine will be in your resource file